Campus Opportunity Radar

AI-Powered Personalized Opportunity Discovery & Recommendation Platform

Project Proposal for UCS503P

Submitted to: **Paramveer kaur**

Thapar Institute of Engineering and Technology

Team Members

Biswajit Mandal – Roll No: 1024030256, Email: bmandal_be24@thapar.edu

Hardik Satija – Roll No: 1024030756, Email: hsatija_be24@thapar.edu

August 13, 2026

1 Higher-order goal

This project aims to reduce the fragmentation and information overload students face while discovering internships, hackathons, competitions, workshops, scholarships, research opportunities, and campus events. The immediate engineering objective is to build a deployable platform that converts scattered opportunity information into structured, personalized, and actionable recommendations.

2 Time-to-value

- Build the core student workflow first: profile, opportunity database, search, filtering, and saving.
- Add AI incrementally for information extraction, eligibility detection, skill extraction, and recommendation explanations.
- Use short development iterations and early user feedback rather than waiting for the complete system.
- Measure success through recommendation quality, extraction accuracy, and time saved in finding relevant opportunities.

3 Problem Statement

Students receive opportunities through college notices, department websites, clubs, WhatsApp and Telegram groups, LinkedIn, Instagram, hackathon platforms, internship websites, and email. This fragmented ecosystem creates several problems:

- **Information overload:** students cannot easily identify which opportunities are relevant.
- **Missed deadlines:** useful opportunities may be discovered after registration closes.
- **Eligibility confusion:** students often spend time checking opportunities for which their branch or semester is not eligible.
- **Skill mismatch:** students may not know which opportunities fit their current skills.
- **No personalization:** most listings show similar results to all students.
- **Poor tracking:** saved opportunities, applications, registrations, and deadlines are not maintained in one place.

The problem is therefore not simply finding opportunities, but identifying the **right opportunities for the right student at the right time**.

4 Proposed Solution

4.1 Overview

We propose **Campus Opportunity Radar**, an AI-powered opportunity intelligence platform. It combines opportunity information with a student's academic profile, skills, interests, eligibility, pref-

erences, and deadlines to generate a personalized opportunity feed.

The system will provide:

- Opportunity collection from controlled campus sources, authorized public sources, APIs/RSS feeds, and manual submissions.
- AI-based extraction of title, category, deadline, eligibility, skills, mode, location, and organizer.
- Student profiles containing branch, semester, skills, interests, and preferences.
- Eligibility filtering and a custom relevance-ranking engine.
- Explainable recommendations showing why an opportunity matches.
- Deadline urgency, saved opportunities, and application/participation tracking.
- Club/organizer submission and administrator approval workflows.

4.2 Core workflow

1. Opportunity is collected from an approved source.
2. The ingestion pipeline processes the announcement.
3. AI extracts structured fields such as skills, eligibility, category, and deadline.
4. The structured opportunity is stored in PostgreSQL.
5. The system compares the opportunity with the student's profile.
6. Eligible opportunities receive a relevance score and are ranked.
7. The student sees a personalized feed with explanations and deadline information.
8. The student can save, register/apply, or track participation.

4.3 Example

For a CSE semester-4 student with Python, Machine Learning, SQL, and React skills and an interest in AI and hackathons, an AI/ML hackathon could be shown as:

> **94% Match**
> Python matches your skills; AI matches your interests; CSE and semester 4 are eligible; campus mode matches your preference; deadline is approaching.

4.4 Operational constraints

The first version will focus on reliable, controlled sources rather than attempting to crawl the entire internet. APIs, RSS feeds, campus submissions, and permitted public pages will be preferred. The system will also be designed for normal student devices and common cloud deployment.

5 Solution Approach

The project focuses on engineering a reliable workflow rather than claiming novel AI research.

5.1 Opportunity intelligence

A raw announcement is converted into structured information:

- Title and category
- Skills required
- Eligible branches and academic level
- Registration deadline

- Duration and mode
- Location and organizer
- Registration URL

Gemini will assist with information extraction, classification, skill extraction, eligibility extraction, and recommendation explanations.

5.2 Matching and ranking

The recommendation engine will use an explicit weighted score:

Component	Weight
Skill Match	30%
Eligibility Match	25%
Interest Match	20%
Deadline Feasibility	15%
Location/Mode Preference	10%

These weights are an initial methodology and will be evaluated using labeled student-opportunity pairs.

5.3 Explainability

Instead of showing only a percentage, the platform will show reasons such as:

- Skills matched.
- Branch and semester eligible.
- Interest matched.
- Location or mode preference matched.
- Deadline approaching.

5.4 Web architecture

The proposed architecture contains:

- **Frontend:** React, TypeScript, Tailwind CSS.
- **Backend:** Python, FastAPI, Pydantic, SQLAlchemy.
- **Database:** PostgreSQL.
- **AI:** Gemini API.
- **Recommendation:** Custom Python scoring and ranking engine.
- **Ingestion:** APIs/RSS, Python HTTP clients, and permitted web extraction.

CI/CD will automatically build, test, lint, and produce repeatable deployment artifacts.

6 Evaluation Criterion

Success will be measured through both system-level and user-level evaluation.

6.1 Primary evaluation metrics

Recommendation Precision@K: measure how many of the top-K recommendations are relevant to a student.

Information Extraction Accuracy: evaluate deadline, eligibility, skill, and category extraction against manually labeled opportunity data.

6.2 Secondary evaluation metrics

- Recall@K and NDCG@K for ranking quality.
- Time required to find five relevant opportunities.
- Recommendation usefulness on a 1–5 scale.
- User satisfaction and ease of discovery.
- Percentage of relevant opportunities discovered before deadlines.

6.3 Pilot validation plan

A pilot can involve approximately 10–20 students. A representative dataset of opportunity announcements will be manually labeled. The personalized system will then be compared with a basic non-personalized opportunity list. User feedback and measured discovery time will be used to evaluate practical usefulness.

7 Scalability

The system should scale from a campus prototype to a broader student opportunity network.

- Separate frontend and backend components.
- Use database indexing and pagination for larger datasets.
- Keep APIs stateless where appropriate.
- Add new opportunity sources without changing the recommendation workflow.
- Support managed PostgreSQL and containerized/cloud deployment.
- Optionally introduce pgvector for semantic matching after the core system is stable.

Initially, the platform can serve one campus and later expand to multiple colleges and universities.

8 Engine availability heuristic

The project relies on mature technologies:

- React and TypeScript for the frontend.
- FastAPI and Python for backend APIs.
- PostgreSQL for structured data.
- Gemini API for AI extraction and explanations.
- SQLAlchemy/Alembic for database operations and migrations.
- Authentication and role-based authorization.
- APIs/RSS and permitted public-source ingestion.
- GitHub Actions or equivalent CI/CD.

These components allow the team to focus on recommendation quality, workflow correctness, and measurable evaluation.

9 Project scope and deliverables

9.1 Initial deliverable

- Student authentication and profile.
- Opportunity database and detail page.

- Search and filtering.
- Save/bookmark functionality.
- Basic CI pipeline and testing.

9.2 Subsequent deliverables

- AI-based opportunity extraction.
- Eligibility and skill extraction.
- Personalized matching and ranking.
- Explainable recommendation feed.
- Deadline alerts.
- Application/participation tracking.
- Club submission portal.
- Admin approval and analytics dashboard.
- Skill-gap and opportunity-trend analysis.

10 Development roadmap

Weeks	Deliverable
1–2	Foundation: React, FastAPI, PostgreSQL, authentication, student profile.
3–4	Opportunity MVP: database, creation, browse, search, filters, save.
5–7	AI extraction: category, deadline, eligibility, and skills.
8–10	Recommendation: matching, ranking, personalized feed, explanations.
11–12	Campus workflow: club portal, admin approval, application tracking.
13–14	Advanced feature: skill gap, trends, or semantic matching.
15	Testing, security, deployment, evaluation, documentation, presentation.

11 Risks and mitigations

- **Incorrect AI extraction:** validate important fields and allow review before publication.
- **Poor recommendations:** use labeled data and compare against a non-personalized baseline.
- **Source limitations:** prioritize campus submissions, APIs, and RSS feeds.
- **Privacy:** minimize stored student information and protect application data.
- **AI/API dependency:** keep the core matching logic independent of the AI service.
- **Scope creep:** complete the MVP before adding advanced features.
- **Security:** protect API keys, hash passwords, enforce roles, and validate inputs.

12 Summary

Campus Opportunity Radar proposes a deployable platform that transforms fragmented student opportunity information into structured, personalized, and actionable recommendations. It combines controlled opportunity ingestion, AI-based information extraction, eligibility filtering, custom relevance ranking, explainable recommendations, and deadline awareness.

The project remains focused on practical engineering: rapid incremental delivery, measurable evaluation, scalable architecture, CI/CD, and a realistic campus pilot. The initial system can start with one campus and later expand to multiple institutions and broader categories of student opportunities.